

オブザーバビリティ成熟度モデル

評価項目	成熟度レベル	説明	具体例
能力成熟度モデル統合 (CMMI)	レベル1: 属人的	プロセスは未定義で場当たり的に運用されており、成功は主に個々の能力や努力に依存している。	-
	レベル2: プロセス確立	プロジェクトレベルで基本的な管理プロセスが確立され、計画・監視・制御によって再現可能な成果が得られている。	-
	レベル3: 組織標準化	組織全体で標準化されたプロセスが定義・文書化され、全プロジェクトで一貫して適用されている。	-
	レベル4: 定量管理	プロセスのパフォーマンスが統計的手法により定量的に測定・制御されており、予測可能かつ安定した成果が得られて	-
	レベル5: 継続的最適化	継続的な改善と技術革新が組織文化として確立され、プロセスの有効性と適応力が継続的に向上している。	-
データ収集と可視化	レベル1	データ収集と可視化は個人の裁量に依存しており、対象範囲や手法は定義されておらず、一貫性もない。記録や手順も未整備で、場当たり的な適用にとどまっている。	サーバのCPU・メモリ使用率のみを監視しており、アプリケーションログは一部ののみ収集している。可視化はシンプルなグラフに限られ、異常検知は手動で行っている。
	レベル2	主要システムに対して、基本的なメトリクスやログの収集・可視化に関する管理プロセスが導入されている。収集手順や品質確認も実施されているが、データの統合やリアルタイム性には課題が残り、プロジェクト単位での実施にとどまっ	メトリクス収集・可視化の手順書を整備し、担当と実施頻度を明確にしている。アプリケーションやネットワークのデータはツールごとに分散されており、更新頻度が低いため即応性に欠けている。データ品質の点検も実施しているが、ツールや手法はチームごとに異っており、一貫性は限定的である。
	レベル3	組織標準として統一されたデータ収集・可視化プロセスが文書化され、インフラ・アプリ・ネットワーク・DB層に一貫して適用されている。可視化基準や品質管理も共通ルールとして定着している。	DatadogやNew Relicなどを使い、全層のモニタリングを標準プロセスに沿って実施している。共通テンプレートで可視化しており、異常発生時の初動確認もルーティン化している。
	レベル4	データ収集・可視化のパフォーマンスが統計的手法により定量的に測定・制御されており、プロセスの品質・精度の安定運用が実現されている。異常検知の精度や検出遅延なども評価対象となっている。	収集率・データ精度・しきい値調整頻度・アラート検出精度などをKPIで管理している。異常検知アルゴリズムの誤検知率・見逃し率を測定しており、継続的にモデルを改善している。
	レベル5	AI・機械学習による予測分析と最適化の仕組みが確立されており、改善提案から導入までが自動化されている。これにより、継続的改善が組織文化として定着している。	AIが収集データからトラフィックの急増を予測しており、ダッシュボードの構成変更やアラートルールの最適化を提案・自動実行している。運用側での介入を最小限に抑えつつ、改善活動を継続的に自動化している。
システムの信頼性管理	レベル1	障害対応は個人の経験と判断に依存しており、対応手順や評価の仕組みは明文化されていない。可用性や性能に関する観測指標も定義されておらず、チームとしての体系的な改善活動は実施されていない。	監視ツールは導入されているが、アラート対応は属人的である。対応履歴や影響範囲の記録が残らず、同様の問題が繰り返されている。システムの可用性や性能に関する目標や基準が未整備であり、観測指標の定義や評価の仕組みも存在していない。システムリスク要因の管理プロセスも確立されていない。
	レベル2	監視ツールや対応手順書を活用した基本的なインシデント管理プロセスが整備されており、プロジェクトレベルで計画的な対応が行われている。移動状況や応答時間など、監視指標や性能指標をもとに改善を試みる仕組みも導入されて	インシデントの記録やエスカレーション手順を文書化し、チーム内で一定のルールに従って対応している。可用性や性能に関する基本的な観測指標(システム稼働率、応答時間など)を明示し、定期的に見直している。
	レベル3	組織全体で標準化されたインシデント対応プロセスおよびシステムリスク要因管理プロセスが整備され、全プロジェクトで一貫して適用されている。信頼性の指標と達成基準が組織共通で整備され、継続的に評価されている。	各チームのSRE担当者が、共通の組織標準プロセス(例:リアルタイムインシデントレポート、ポストモートミー)に基づき、インシデントの検知・対応・復旧・振り返りを実施している。可用性やエラーレートなどの観測指標に基準値を設定し、定期的にレビューしている。プレイブックによる対応標準化や、システムリスク要因の優先順位付け・改善計画も整備されている。
	レベル4	観測された信頼性指標(例:稼働率、障害件数、対応時間など)をモニタリングし、対応プロセスやリスク要因削減の成果を統計的に評価している。予兆検知や事後レポート連携、改善を管理している。	収集された稼働データをもとに、再発防止策や対応遅延の改善効果などを評価している。予兆検知の仕組みにより、障害の兆候を早期に特定できるようにしており、CI/CDパイプラインにも信頼性のチェックを組み込んでいる。
	レベル5	観測データのAI・機械学習による高度な分析が自律的に実行されており、障害予兆の検知から対応まで全自動化されている。継続的な改善が組織文化として定着し、人的介入を最小限に抑えた予防的な信頼性向上が実現されている。	AIがログ・メトリクスを解析して障害予兆を検知しており、インシデント種別に応じた復旧手順を自動実行している。パフォーマンス異常や障害予兆の分析・対応を全自動化し、対応記録やナレッジも自動的に蓄積・共有している。サービス影響を抑えながら、継続的な改善もAIにより自律的に実現している。
開発・運用プロセスの整備と最適化	レベル1	コード品質の基準やレビュー体制が整備されておらず、開発およびリリースは属人的かつ場当たり的に実施されている。	開発者とコードの書き方が異なっており、統一されたコーディング規約や品質基準が存在していない。テスト工程はほとんど実施されておらず、バグが頻発している。リリースは計画性に欠けており、突発的に実行されることが多い。
	レベル2	静的解析やコードレビューを導入し、一部の品質管理が実施されている。リリースも一定のスケジュールで管理されているが、運用にはばらつきがある。	静的解析ツールを活用して最低限の品質チェックを自動化している。コードレビューを実施してバグの抑制に取り組んでいるが、レビュープロセスには一貫性がない。リリースは定期的に行っているものの、品質保証体制が不十分でスケジュールの遅延が発生している。
	レベル3	CI/CDパイプラインが整備され、自動テストやステージング環境を活用して品質と本番環境の安定性を確保している。	コードの変更がマージされると、単体テスト・統合テスト・リグレッションテストが自動で実行される仕組みを整備している。本番環境へのデプロイ前には影響評価も実施しており、リリースに伴う障害リスクを抑制している。
	レベル4	コード品質やリリースの結果をKPIとして定量的に測定し、影響範囲の可視化とフィードバックループを通じて継続的な改善を実施している。	コードカバレッジ、エラー率、デプロイ頻度などのKPIを設定し、定期的なレビューを通じて品質向上に取り組んでいる。本番環境の挙動をリアルタイムで監視し、リリース後の影響を即時に分析して、必要に応じてロールバックや修正リリースを実施している。
	レベル5	AIによるコード解析と変更影響分析により、リスクやバグを自動で検出し、レビューからデプロイまでのプロセスを完全自動化している。	AIがコードの変更点を解析し、テスト結果や過去の障害データをもとにリリース可否を自動で判断している。問題があれば修正案を提示し、必要に応じて開発者が対応できる体制を整備している。リリース後もシステム挙動を監視し、自動ロールバックなどの復旧処理を即時に実行して、安定した運用を実現している。
アラート最適化と障害対応	レベル1	固定しきい値のアラートのみで運用されており、誤検知・見逃しが多く、障害対応も手動かつ反応的に実施されている。	CPUやメモリ使用率の固定しきい値を設定し、閾値を超えた場合にアラートが発報されている。しかし、負荷の変動を考慮できず誤検知が多発している。担当者が手動で対応しており、不要なアラート処理が日常的に発生している。
	レベル2	アラートの重要度や対象を整理し、しきい値調整などによるノイズ削減に取り組んでいるが、改善効果は限定的である。	アラートの優先度を定義し、しきい値を調整することで通知量を削減しているが、負荷の急変には対応できず、ピーク時には誤検知や遅延が発生している。その結果、重要な異常の見逃しや対応の遅れが頻っている。
	レベル3	動的しきい値や異常検知の仕組みを導入し、アラートの精度を向上させ、不要な通知を削減している。	CPU・メモリ・ネットワークトラフィックなどの過去データを学習し、通常の変動範囲を自動で判断する異常検知アルゴリズムを導入している。これにより、しきい値の手動調整が不要になり、アラートノイズの大幅な削減を実現している。
	レベル4	アラート履歴や影響度をもとに、AIによる優先度付けや自動チューニングを行い、判断支援と迅速な対応を実現している。	AIが過去のアラート発生パターンを学習し、誤検知を抑制しながらクリティカルな異常の優先通知を実施している。これにより、エンジニアの認知負荷を軽減し、迅速かつ的確な障害対応を可能としている。
	レベル5	異常検知から原因推定、対処アクションの実行までを自動化し、オペレーターの介入なしでアラート処理を完結させている。	AIがリアルタイムで異常を検知し、事前に定義された対応策(スケールアウト、トラフィックのリルーティング、自動ロールバックなど)に基づいて最適なアクションを判断・実行している。これにより、システムは人的介入なしで自己回復し、安定稼働を継続できるようにしている。
ユーザー行動の理解と最適化	レベル1	ユーザーの行動ログをほとんど収集しておらず、分析も実施されていないため、製品改善に活用されていない。	アクセスログやイベントデータの取得が限定的であり、ユーザーの行動分析を行っていない。そのため、改善施策の立案は担当者の経験や勘に依存している。
	レベル2	ページビューやクリックなどの基本的な行動データを収集・可視化し、定量的な指標に基づく意思決定を開始している。	Google Analyticsやアプリ内の簡易なランキングを活用し、PV・UV・滞在時間などの基本的な行動指標を分析している。ただし、スクロール深度・クリック率・遷移/リターンなどの詳細データは収集しておらず、具体的な改善に十分活かされていない。
	レベル3	ユーザー行動データをKPIとして活用し、継続的な機能改善やUX最適化を実施する仕組みが確立されている。	コバートラン率・セッション数・セッション継続時間・アクティブユーザー数(DAU/WAU/MAU)・ユーザーフローなどをKPIとして設定し、定期的に分析している。これらのデータに基づき、機能改善やUI調整を継続的に実施している。
	レベル4	機械学習を活用してユーザー属性や行動パターンを分類・予測し、リアルタイムでUIや機能を動的に最適化している。	AIがユーザーの過去の行動データを分析し、リアルタイムでパーソナライズされたコンテンツやレコメンデーションを提供している。たとえば、閲覧履歴に応じて最適なコンテンツを自動で推薦し、エンゲージメントやコンバージョン率の向上につなげている。
	レベル5	行動分析から改善提案・施策適用までのフィードバックループが自動化されており、ユーザー体験の最適化が継続的かつ動的に行われている。	AIがユーザーの行動をリアルタイムで分析し、A/Bテストの結果をもとにUIや機能の最適化を自動で実行している。たとえば、個々の行動パターンに応じて重要な機能を強調表示したり、使用頻度の低い機能を非表示にすることで、常に最適な体験を提供できる状態を整備している。
継続的な改善と最適化	レベル1	改善活動は担当者の裁量に任されており、仕組み化されていない属人的な対応にとどまっている。	開発や運用の課題が発生しても、特定の担当者が個別に対応しており、組織的なフィードバックループが存在していない。過去の知見が蓄積されず、同じ問題を繰り返している。
	レベル2	レビューやふりかえりを定期的に実施し、課題や改善点をチームで共有する文化が形成されつつある。	開発チームが定期的な振り返りミーティングを実施し、プロセス上の課題を共有している。しかし、改善施策の優先順位が明確でなく、具体的なアクションに落とし込めておらず、同じ課題が繰り返し議論されている。
	レベル3	チーム全体で改善プロセスが明文化・体系化されており、継続的改善がプロジェクトの一部として定着している。	開発速度、バグ修正率、デプロイ頻度などのKPIを設定し、その成果を定量的に測定している。これらの仕組みが組織として整備されており、継続的なプロセス改善を推進する体制を構築している。
	レベル4	改善指標(KPI)やログデータを活用し、改善サイクル(PDCA)が測定・制御されており、成果も定量的に評価されている。	開発や運用のモニタリングデータを分析し、問題発生時には自動で改善ポイントを特定する仕組みを構築している。改善施策を定期的に実施し、最適化を標準業務として継続的に実行している。
	レベル5	AIが改善対象やアクションを自動で特定・提案・実行し、最適なプロセス改善が自律的に実施されている。	AIが開発・運用データをリアルタイムで分析し、ボトルネック作業や非効率なプロセスを自動で特定している。リソース配分の最適化や不要な作業の削減を自動で行い、生産性向上やエラー削減に寄与する施策を人手を介さずに継続的に実施している。